

Advanced signal processing

Exam 2

Name	Initials	Student number	Retning
Alexander Nygaard Thomsen	AT	201911329	E

Due date: 12/11/2026

1 Introduction

This lab journal will take a shallow dive in to adaptive signal processing, specifically comparison of three common types of implementation, the regular *LMS*(REF), the *leaky-LMS*(REF) and the *Variable step-size LMS*(REF). Comparison of their performances in an experimental sense, a delve into their specifics, use cases and a critical discussion.
It is a assumed that the reader has some basic understand of digital filtering and a general grasp of their implementation, therefore a minimum amount of effort will be spend on that topic, however characterics regarding these mentioned types of filters, will be explained and discussed.
Filtering in general, is the art of removing the unwanted or highlighting and improving specific feature of input signals. Mainly in the analog domain, a filter would be designed for given frequency or bandwidth, with little possibility of expanding or changing. Such is the same problem when designing FIR(REF) or IIR(REF) filters in the digital domains. The lack of ability to adapt the characterics of the filtering gave birth to the aptly named *Adaptive filter*.

2 Theoretical background

Contrary to regular filtering, the designer targets a specific frequency or bandwidth, placing zeroes and poles on the unit-circle to *approximate*, a desired result, or atleast get somewhat close to something acceptable.

In adaptive filtering, we're more concerned about the system as an input and how we implement an algorithm to force the error to be zero. In this lab journal the overall system we'll be examining will take the form of:

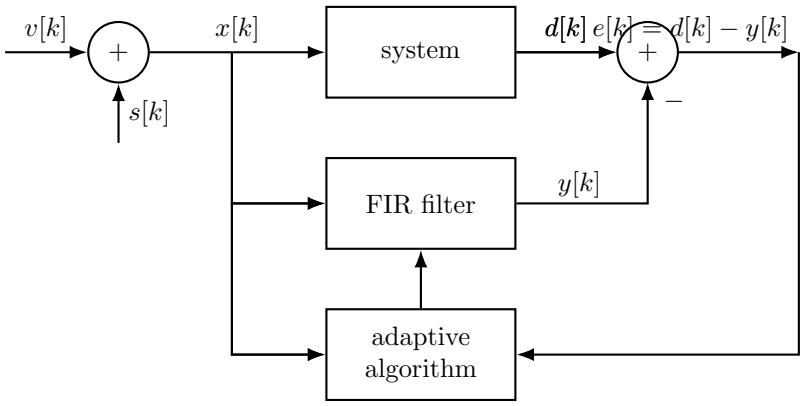


Figure 1: Adaptive FIR filter structure.

Where $x[k]$ will be some randomly generated noise, with a zero-mean, unit variance, passing through a system summed together with a sinusoid $s[k]$, for testing purposes, to experimentally see how the system responds to both noise and a signal combined.

In this example, the unidentified system will be $d[k] = 0.5x[k] + 0.25x[k-1] - 0.3x[k-2]$

The simple LMS filter works by taking the output of the system, as the desired output $d[k]$ and then subtracting $y[k]$, which is the output of the FIR filter, governed by a vector of weights $w[k]$. The The LMS algorithm will then try to force the error $e[k] = d[k] - y[k] \approx 0$ by tuning the weights in descrete steps.
The vector $\mathbf{W}[k]$ hold the coefficients and the vector $\mathbf{X}[k]$ holds old samples :

$$\mathbf{W}[k] = \begin{bmatrix} w_0[k] \\ w_1[k] \\ \dots \\ w_N - 1[k] \end{bmatrix}, \qquad \mathbf{X}[k] = \begin{bmatrix} x[k] \\ x[k-1] \\ \dots \\ x[k-N+1] \end{bmatrix}$$

The complete algorithm for computing the coefficiens for the vector:

$$\mathbf{W}(k+1) = \mathbf{W}[k] + 2\mu e[k]\mathbf{x}[k]$$

The factor μ is factor that gives some control of how aggressize the filter should act, as it a scalar that is multiplied onto the weights and the old samples. The idea of the algorithm is then, that each iteration that it runs, the weights of the filter are multipled with the signal, which affects how $y[k+1]$ samples will behave, this affects the error, and it repeats. Each iteration the weights gets updates aswell, so that they are applied with a different value at the next iteration.

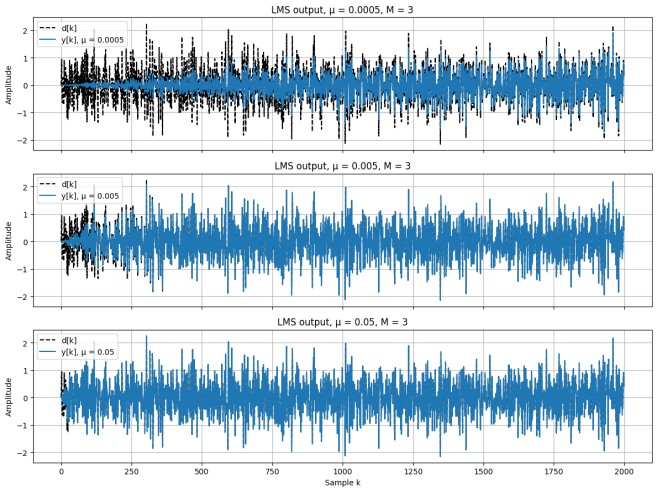
3 Numerical experiment

3.1 LMS experimentation

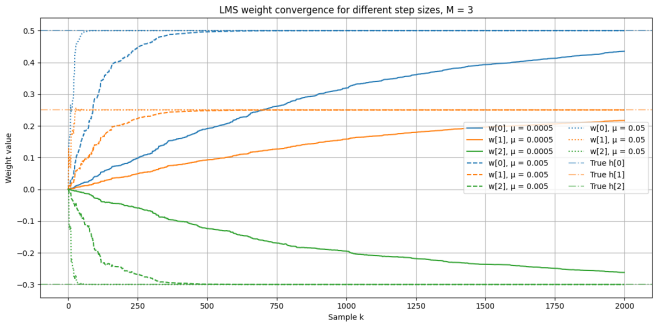
Looking at the equations for the LMS filter, we're given some handles to tune the filter with namely μ, M the amount of filter taps and N how long the algorithm will run.

For the initial tests, we'll keep the amount of filter tabs fixed to $M = 3$, and we'll instead vary the step size by a factor of 1000, and keep the test running for a 2000 samples. This is an estimate to have enought samples to atleast at higher values for μ that the filter will converge.

The next experiment is increasing the filter taps, where $M = [3, 5, 10]$. This is done to observe if increasing the filter tabs, beyond the amount of poles present in the unidentified system, has any influence on either convergence speed and accuracy.

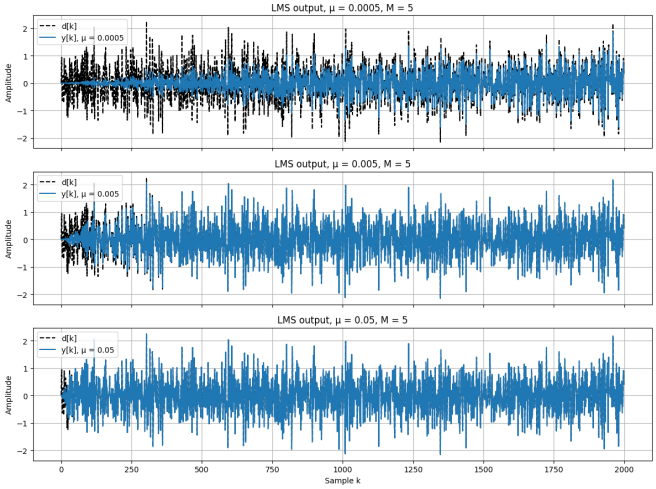


(a) LMS, M=3

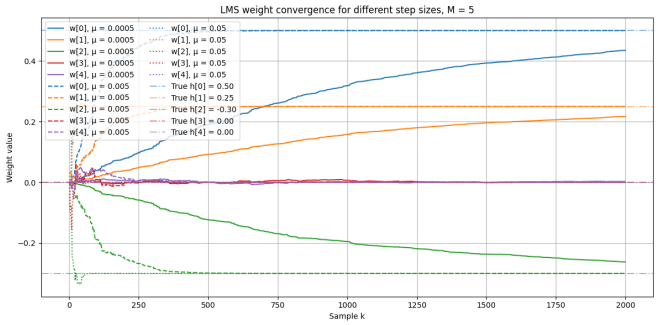


(b) LMS, weights, M=3

Figure 2: Comparison of sizes of μ for M=3

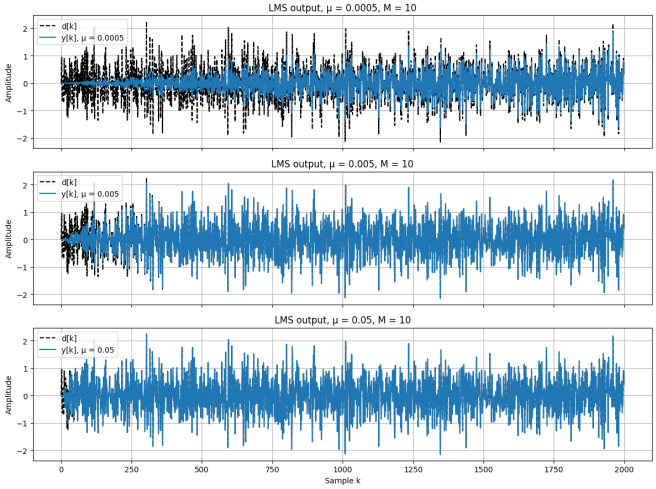


(a) LMS, M=5

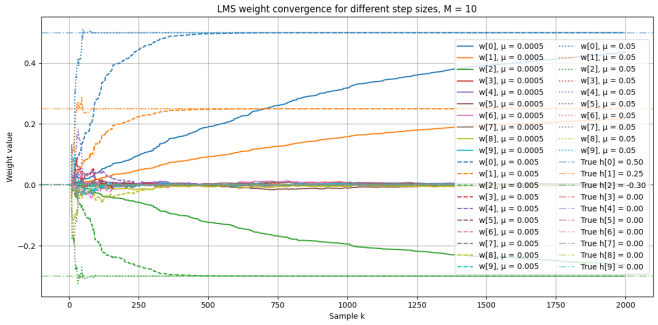


(b) LMS, weights, M=5

Figure 3: Comparison of sizes of μ for M=5



(a) LMS, M=10



(b) LMS, weights, M=10

Figure 4: Comparison of sizes of μ for M=10

3.2 leaky LMS experimentation

3.3 variable step-size experimentation

4 Results

4.1 Analysis

4.1.1 LMS

Starting with the output of the regular LMS filter, it can be seen on figure:2, where the size of the μ is very small and how it compares to an increase in size in an order og magitude twice. Towards the end of the 2000 samples, $y[k]$ seems to almost be equal to that of $d[k]$ meaning that the error, which is the difference of two would become zero, this would essentailly mean that the filter that converged on the weights of the unidentified system. However as the value of μ

increases, the speed at which this happens increases, quite dramatically.

Taking a look at the graph for the weights, the semi dashed lines, belonging to $\mu = 0.005$, the weights have almost converged after 500 samples, which is a fourth of the total amount of samples. And again, an increases in this speed, or decrease in time, can be see if the value is even larger. Looking at the equation for the update of the weights, μ is essentailly just scaling how much we want the product of the error and the old samples, to affect the future weights. As this goes on for every iteration, it becomes intuitive how it slows the entire process down.

We can consider an example where we let every input sample be $x[k] = 1$ and we have only the first term in our unidentified system, difference equation $d[k] = 0.5x[k]$, and the filter is initialized to be $w[0] = 0$, we want to find the $w_{ideal} = 0.5$ such that the error is forced to 0. A entire iteration with $\mu = 0.05$ of the LMS becomes:

$$d[k] = 0.5(1) = 0.5$$

$$y[0] = w[k]x[k] = 0(1) = 0$$

$$e[k] = 0.5 - 0 = 0.5$$

$$w[k+1] = w[k] + 2\mu e(k)x(k) = 0 + 2(0.05)(0.5)(1) = 0.05$$

Table 1: comparison of LMS weight update

k	$\mu = 0.05$	$\mu = 0.4$
0	0.0000	0.0000
1	0.0500	0.4000
2	0.0950	0.4800
3	0.1355	0.4960
4	0.1720	0.4992
5	0.2048	0.4998
6	0.2341	0.4999
7	0.2609	0.4999
8	0.2848	0.4999
9	0.3063	0.5000
10	0.3257	0.5000

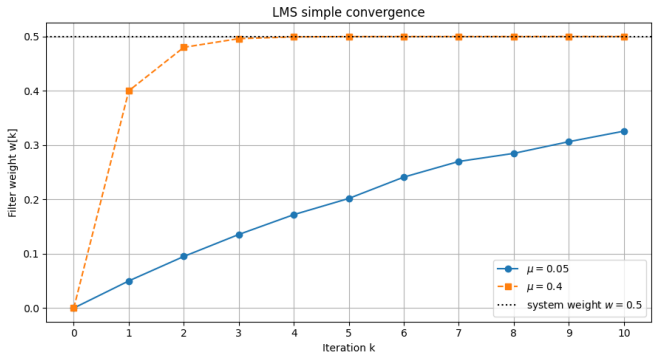


Figure 5: Simple converge

What table 1 shows is that with the large step size, we're already 80% of the way towards the convergence after the first iteration, but getting closer to the ideal weight become harder. In this particular case, the convergence is nicely approaching the ideal coefficients, this is due to the constant input of 1. However if the system gets a sudden impulses added to the input, then it is easy to see that the larger value of μ wont *dampen* the larger step input, but larger values of μ also comes with some drawbacks that will be expanded upon in the discussion section.

4.2 Discussion

As alluded to in the LMS analysis section, larger values of μ is not necessary a good thing. If too a large is selected, the resulting product of the input and the error is essentailly gained by the value, if this value $\mu \leq 1$ it is easy to see that, this will cause the resulting calculations to start spiralling out of control. In the simple example given ealier, an equation for the behaviour of the oscillations can be derived.

$$w[k+1] = w[k] + 2\mu e[k]x[k]$$

In this case, the single tab, the desired signal can be described as: $d[k] = \alpha x[k]$ and filter $y[k] = w[k]x[k]$ and the error $e[k] = d[k] - y[k]$ substituting $d[k]$ and $y[k]$:

$$e[k] \Rightarrow \alpha x[k] - w[k]x[k] = (\alpha - w[k])x[k]$$

Substituting this into the update equation yields:

$$w[k+1] = w[k] + 2\mu(\alpha - w[k])x^2[k]$$

Manipulating the equation and in this case x[k] = 1:

$$w[k+1] = w[k] + 2\mu\alpha - 2\mu w[k]$$

$$w[k+1] = w[k] - 2\mu w[k] + 2\mu\alpha$$

$$w[k+1] = (1 - 2\mu)w[k] + 2\mu\alpha$$

5 Conclusion